GODSHAND — Creator Guide (Story, Voice, Music & Multi-Character)

🎬 Two inputs only: a STORY_YAML block and one image per character. From those, the kit renders the video, voices the dialogue, and scores the music — no audio files needed.

What you provide vs. what the kit makes

You provide	The kit generates
STORY_YAML — story, shot list, spoken `line:` text, and `song:` mood labels (all text)	Video from character anchors + each shot's `action:` (Wan image-to-video)
One face image per character	Voice — Kokoro TTS reads the `line:` text you typed in the YAML
(nothing else)	Music — MusicGen composes a bed from the `song:` label, auto-ducked under the voice

The one seam: STORY_YAML

The setup cell has a single variable, `STORY_YAML`. On run it compiles:

```
STORY_YAML → _build_shots_from_story() → shots_confrontation.json → render loop
```

To change the film, you edit only this block. Nothing else.

Top-level keys

Key	What it is
`film`	title string
`note`	one-line logline
`look`	global style, applied to every shot unless a shot overrides it
`avoid`	global negative prompt
`characters`	map of name → `{ look, anchor }`
`shots`	the list — each entry is one render unit

Do not add a `render:` block — the validated Wan quality settings live in the notebook config, and overriding them risks the melt/rainbow regression.

Shot fields

Field	Meaning
`id`	unique string, e.g. `scene01_shot_03`
`secs`	duration in seconds (default 4)
`who`	list of character names (must match `characters` keys)
`camera`	camera move, e.g. "slow push-in", "low-angle crane up"
`action`	the main description: subject + emotion + camera, ending with the anti-melt tail
`line`	the spoken words (also accepts `say:` / `dialogue:`)
`talking` / `voiceover`	booleans
`song`	music mood label (shots sharing a label share one bed)
`palette`	color mood
`look`	per-shot style override (optional)
`takes`	optional, overrides rejection_samples for that shot

Every `action:` must end with the anti-melt tail — it is what holds the image together: `smooth steady camera movement, gentle natural motion, subtle drifting atmosphere and soft shifting light, stable coherent shapes, solid consistent forms, a living breathing cinematic shot, no freeze`

Generating a story with an LLM (Grok)

Paste your prose story into this prompt and drop the YAML it returns into `STORY_YAML`:

```
You are a shot-list compiler for an image-to-video AI film kit.
I will give you a story in prose. Convert it into YAML in EXACTLY this schema.

Rules:
- Output ONLY valid YAML, no commentary, no code fences.
- Top-level keys: film, note, look, avoid, characters, shots.
- characters: give each person a name, a `look:` description, and an `anchor:`
  filename like name_anchor.png. Reuse jack (man_anchor.png) and julie
  (woman_anchor.png) for the two shipped faces.
- Every shot needs a UNIQUE id like scene01_shot_01.
- secs: 4 (keep all shots 4 seconds).
- who: list only character names defined above.
- Put spoken dialogue in `line:` and set talking: true (or voiceover: true).
- Give each shot a `song:` mood label; reuse the same label across a scene.
- action: describe the moment cinematically (subject + emotion + camera move),
  then ALWAYS end the action with this exact tail:
  "smooth steady camera movement, gentle natural motion, subtle drifting
  atmosphere and soft shifting light, stable coherent shapes, solid consistent
  forms, a living breathing cinematic shot, no freeze"
- Do NOT include a `render:` block. Do NOT change resolution/steps/model.

Here is the story:
<PASTE YOUR STORY HERE>
```

Voice + music

- Voice — Kokoro TTS (English), set up in the audio step. It reads the `line:` text from each shot. Voice is synthetic narration/dialogue, not a specific person's cloned voice.
- Music — MusicGen (`facebook/musicgen-medium`), keyed by the per-shot `song:` label. Shots sharing a label get one continuous bed. It is a generated instrumental score, not a licensed track.
- Mixing — `stitch.py` ducks music to −4dB under the voice and fixes the stereo layout automatically.
- Tip — for a clean demo, use one or two `song:` labels for the whole piece rather than a new one per shot.

Multi-character support (patched notebook)

The patched notebook (`rainman-wan-multichar.ipynb`) adds two things:

1. N-character ingestion — after the jack/julie bind, it loops over every character in `STORY_YAML` that declares an `anchor:`, finds that image anywhere under `/kaggle/input`, and installs it to `refs/<name>/face/` so it is identity-locked. Characters with no matching image fall back to text-forge (unchanged behavior). The original 2-character path is untouched.
2. Spoken-line fields — every shot now carries `line` / `vo_text` into `shots_confrontation.json`.

To add a character: give it a name in `characters:` with a `look:` and an `anchor:` filename, then upload an image with that exact filename (e.g. `elder_anchor.png`) to the Kaggle dataset. No image = text-forged from the look tokens.

Example: THE LANTERN KEEPER (runs as-is)

Validated against the real patched builder — 3 characters, dialogue, and music all parse through:

```
film: THE LANTERN KEEPER
note: Demo - multi-character story with voice and music. Runs
on the kit as-is.

look: warm analog neon twilight, soft synth-wave glow, magent
a and amber haze, gentle grain, nostalgic hush, 35mm
avoid: warped faces, extra fingers, duplicate limbs, morphin
g, flicker, text, watermark, jitter, plastic skin

characters:
  jack:
    anchor: man_anchor.png
    look: a weary man in his forties, olive field jacket over
a white tee, short dark hair, tired honest eyes, faint stubbl
e
  julie:
```

```
    anchor: woman_anchor.png
    look: a calm luminous woman, soft dark hair, gentle steadfast gaze, pale scarf, quiet warmth like an angel in human form
  elder:
    anchor: elder_anchor.png
    look: an old lantern keeper, deep-lined kind face, silver beard, worn brown coat, holding a small brass lantern, patient eyes

shots:
- id: lantern_01
  secs: 4
  who: [jack]
  camera: slow push-in
  song: prelude
  talking: true
  line: I kept the light because someone had to.
  palette: amber lamp glow
  action: a slow push-in on the weary man standing in a dim doorway holding a small brass lantern, speaking softly to himself, warm lamplight on his face, smooth steady camera movement, gentle natural motion, subtle drifting atmosphere and soft shifting light, stable coherent shapes, solid consistent forms, a living breathing cinematic shot, no freeze

- id: lantern_02
  secs: 4
  who: [jack, julie]
  camera: slow orbit
  song: prelude
  talking: true
  line: You never had to carry it alone.
  palette: magenta and amber haze
  action: a slow orbit around the man and the luminous woman facing each other in the neon haze as she reaches toward the lantern between them, tender recognition, smooth steady camer
```

```
a movement, gentle natural motion, subtle drifting atmosphere
and soft shifting light, stable coherent shapes, solid consis
tent forms, a living breathing cinematic shot, no freeze

- id: lantern_03
  secs: 4
  who: [elder]
  camera: low-angle crane up
  song: hymn
  voiceover: true
  talking: false
  line: Every keeper passes the flame; none of us owns it.
  palette: quiet neon fade
  action: a low-angle crane up on the old lantern keeper rais
ing his brass lantern into the twilight as embers drift upwar
d, quiet reverence, smooth steady camera movement, gentle nat
ural motion, subtle drifting atmosphere and soft shifting lig
ht, stable coherent shapes, solid consistent forms, a living
breathing cinematic shot, no freeze
```

Creator workflow

1. Write or generate the story (use the Grok prompt above), defining each character in `characters:`.
2. Upload one face image per character to the Kaggle dataset, named to match each `anchor:` (jack/julie use `man_anchor.png` / `woman_anchor.png`).
3. Paste the YAML into `STORY_YAML`. Add `line:` + `talking`/`voiceover` for speech and a `song:` label for mood.
4. Run the setup cell — the kit compiles the shots and reports which characters are identity-locked vs text-forged.
5. Render a short chunk (`KIT_CHUNK_SIZE` small), inspect frames, then scale.
6. Package — the stitched clip + zip drop out automatically.

Honest limits (don't oversell)

- Voice is synthetic TTS (Kokoro, English) — not a specific person's cloned voice unless voice-cloning is wired later.
- Music is a generated instrumental bed — mood-matched, not a bespoke human score or licensed song.
- Lip-sync is approximate — `talking: true` drives mouth motion but is not frame-perfect to the words. Good for cinematic VO/ambient dialogue, not a talking-head substitute.
- Voice last-mile — the `line:` text is captured into the shots JSON, but Kokoro actually speaking it depends on one wire in `src/render_motion.py`, confirmed on the next live run. Music and video work today.
- Multi-character-in-one-frame is the hard case for any image-to-video engine — identity holds strongest for the character whose anchor seeds a shot.

For demo recording

Show the loop small: 2–3 characters, 2–3 shots, one `song:` label, one spoken `line:`. That demonstrates the three selling points live — your own cast, your own story, voice + music baked in — without a long render wait. Say the "scales to a full cast and feature length" claim; show the short multi-character clip.